



KHULNA UNIVERSITY OF ENGINEERING & TECHNOLOGY

Course No: CSE 2113

Course Title: Computer Architecture Laboratory

Project : 22-Bit Minimal Computer System Design Using Logisim

Submitted by: Miah Tahsin Ibna Mezan

Roll: 2307053

Submitted to:

Md. Badiuzzaman Shuvo Lecturer Department of Computer Science and Engineering Khulna University of Engineering & Technology	Mubtashim Abrar Nihal Lecturer Department of Computer Science and Engineering Khulna University of Engineering & Technology
---	---

1. Introduction

This report presents the design and implementation of a minimal computer system using Logisim simulation software based on a microprogrammed control architecture. The word size of the system is 22 bits, as assigned, and the design supports a minimum of eight instructions including a complex multiplication instruction implemented using Booth's algorithm.

The computer system consists of a Central Processing Unit (CPU) and a Main Memory (RAM). The CPU contains a microprogrammed Control Unit (CU), an Arithmetic Logic Unit (ALU), and a set of registers. All components are designed and simulated within Logisim, organized into a clean hierarchical subcircuit structure.

2. System Architecture Overview

The overall system architecture follows a single-accumulator, memory-reference design. The top-level view of the system contains only three components: a Clock, the CPU subcircuit, and the RAM component. This keeps the top-level view clean and professional, with all internal complexity encapsulated within the CPU subcircuit.

Table-2.1: Top-Level Structure

Component	Type	Description
Clock	Wiring	System clock driving CPU and RAM
CPU	Subcircuit	Contains all processing logic
RAM	Memory	11-bit address, 22-bit data width (2048 locations)
Reset	Input Pin	Resets PC and Accumulator to zero

Table-2.2: CPU Internal Structure

Component	Width	Function
PC (Program Counter)	11-bit	Holds address of next instruction
MAR (Memory Address Register)	11-bit	Holds address sent to RAM
IR (Instruction Register)	22-bit	Holds current fetched instruction
A (Accumulator)	22-bit	Main data register for ALU results
B (Memory Buffer Register)	22-bit	Temporary buffer for memory data
ALU	22-bit	Performs AND, OR, ADD, SUB operations
Control Unit	Subcircuit	Microprogrammed ROM-based controller

3. Instruction Set Architecture

Table-3.1: Instruction Word Format

Each instruction is 22 bits wide, divided as follows:

Bits	Field	Width	Description
21 $\div$ 17	OPCODE	5 bits	Identifies the instruction (up to 32 instructions)
16 $\div$ 11	Unused	6 bits	Reserved / not used
10 $\div$ 0	ADDRESS	11 bits	Memory address operand (2048 locations)

Table-3.2: Instruction Set

The system supports 9 instructions, exceeding the minimum requirement of 6:

Mnemonic	Opcode (5-bit)	Operation	Description
AND	00000	$A \leftarrow A \wedge \text{Mem}[\text{addr}]$	Bitwise AND with memory
ADD	00001	$A \leftarrow A + \text{Mem}[\text{addr}]$	Add memory to accumulator
STO	00010	$\text{Mem}[\text{addr}] \leftarrow A$	Store accumulator to memory
OR	00011	$A \leftarrow A \vee \text{Mem}[\text{addr}]$	Bitwise OR with memory
SUB	00100	$A \leftarrow A \text{ } \bar{\wedge} \text{Mem}[\text{addr}]$	Subtract memory from accumulator
MOV	00101	$A \leftarrow B$	Move B register to accumulator
BUN	00110	$\text{PC} \leftarrow \text{addr}$	Branch unconditionally
LDA	00111	$A \leftarrow \text{Mem}[\text{addr}]$	Load memory into accumulator
MUL	01000	$A \leftarrow A * \text{Mem}[\text{addr}]$	Multiply using Booth's algorithm
HLT	11111	Halt	Stop the clock

4. Microprogrammed Control Unit

The Control Unit is implemented as a microprogrammed controller following the architecture shown in Figure 10-11 of the support material (Microprogram control for processor unit). It uses a Control Memory (ROM) to store microinstructions, a Control Address Register (CAR) to sequence through them, and a MUX to select the next microinstruction address.

Table-4.1: Control Word Format

Each control word stored in the ROM is 22 bits wide:

Bits	Field	Width	Description
21 ÷ 18	F	4 bits	Microoperation select (decoded by 4-to-16 decoder)
17 ÷ 15	Unused	3 bits	Reserved
14 ÷ 13	CD	2 bits	Condition select (unused, set to 00)
12 ÷ 11	BR	2 bits	Branch type: 00=fetch, 01=increment, 10=branch, 11=decode
10 ÷ 0	AD	11 bits	Branch address field

Table-4.2: F Field Microoperation Encoding

The 4-bit F field is decoded by a 4-to-16 decoder inside the CPU to generate individual control signals:

F Value	Decoder Output	Microoperation	Registers Affected
0000	0	None	∅
0001	1	$MAR \leftarrow PC$	MAR
0010	2	$PC \leftarrow PC + 1$	PC
0011	3	$B \leftarrow M \text{ (MemRead)}$	B, IR
0100	4	$PC \leftarrow B(AD)$	PC
0101	5	$MAR \leftarrow B(AD)$	MAR
0110	6	$A \leftarrow A \wedge B$	A (ALU AND)
0111	7	$A \leftarrow A + B$	A (ALU ADD)
1000	8	$A \leftarrow A \vee B$	A (ALU OR)

1001	9	$A \leftarrow A \bar{\wedge} B$	A (ALU SUB)
1010	10	$B \leftarrow A$	B
1011	11	$M \leftarrow B$ (MemWrite)	RAM
1100	12	$A \leftarrow B$	A
1101	13	MUL (Booth's enable)	A
1111	15	HLT	Clock stopped

4.3 Microprogram Sequencing

The CAR sequences through microinstructions as follows:

- BR = 00: $CAR \leftarrow 0$ (return to FETCH cycle)
- BR = 01: $CAR \leftarrow CAR + 1$ (increment to next microinstruction)
- BR = 10: $CAR \leftarrow AD$ (direct branch to address)
- BR = 11: $CAR \leftarrow 4 * opcode + 3$ (decode jump to instruction routine)

Table-4.4: Microprogram Table

The complete microprogram stored in the Control ROM (addresses 0000-3F hex):

ROM Addr	Hex Value	F	BR	Microoperation	Next
00	040800	0001	01	$MAR \leftarrow PC$	CAR+1
01	0C0800	0011	01	$B \leftarrow M$	CAR+1
02	081800	0010	11	$PC \leftarrow PC+1, IR \leftarrow B$	Decode
03	140800	0101	01	AND: $MAR \leftarrow B(AD)$	CAR+1
04	0C0800	0011	01	$B \leftarrow M$	CAR+1
05	180800	0110	01	$A \leftarrow A \wedge B$	CAR+1
06	000000	0000	00	None	$CAR \leftarrow 0$
07	140800	0101	01	ADD: $MAR \leftarrow B(AD)$	CAR+1
08	0C0800	0011	01	$B \leftarrow M$	CAR+1
09	1C0800	0111	01	$A \leftarrow A + B$	CAR+1
0A	000000	0000	00	None	$CAR \leftarrow 0$

0B	140800	0101	01	STO: MAR $\leftarrow$ B(AD)	CAR+1
0C	280800	1010	01	B $\leftarrow$ A	CAR+1
0D	2C0800	1011	01	M $\leftarrow$ B	CAR+1
0E	000000	0000	00	None	CAR $\leftarrow$ 0
0F	140800	0101	01	OR: MAR $\leftarrow$ B(AD)	CAR+1
10	0C0800	0011	01	B $\leftarrow$ M	CAR+1
11	200800	1000	01	A $\leftarrow$ A $\vee$ B	CAR+1
12	000000	0000	00	None	CAR $\leftarrow$ 0
13	140800	0101	01	SUB: MAR $\leftarrow$ B(AD)	CAR+1
14	0C0800	0011	01	B $\leftarrow$ M	CAR+1
15	240800	1001	01	A $\leftarrow$ A ∇ B	CAR+1
16	000000	0000	00	None	CAR $\leftarrow$ 0
17	300800	1100	01	MOV: A $\leftarrow$ B	CAR+1
18	000000	0000	00	None	CAR $\leftarrow$ 0
1B	100800	0100	01	BUN: PC $\leftarrow$ B(AD)	CAR+1
1C	000000	0000	00	None	CAR $\leftarrow$ 0
1F	140800	0101	01	LDA: MAR $\leftarrow$ B(AD)	CAR+1
20	0C0800	0011	01	B $\leftarrow$ M	CAR+1
21	300800	1100	01	A $\leftarrow$ B	CAR+1
22	000000	0000	00	None	CAR $\leftarrow$ 0
23	140800	0101	01	MUL: MAR $\leftarrow$ B(AD)	CAR+1
24	0C0800	0011	01	B $\leftarrow$ M	CAR+1
25	340800	1101	01	Booth's Enable	CAR+1
26	000000	0000	00	None	CAR $\leftarrow$ 0
3F	3C0000	1111	00	HLT	CAR $\leftarrow$ 0

5. Datapath Design

The datapath connects all registers and functional units. Control signals from the microprogrammed CU activate specific paths on each clock cycle.

Table-5.1: Register Transfer Summary

Step	Microoperation	F Value	Signal Generated
MAR load from PC	$MAR \leftarrow PC$	0001	MAR_enable, MUX selects PC
Memory read	$B \leftarrow M$	0011	MemRead=1, B_enable, IR_enable
PC increment	$PC \leftarrow PC+1$	0010	PC_enable, MUX selects adder
MAR load from IR	$MAR \leftarrow B(AD)$	0101	MAR_enable, MUX selects IR[10:0]
ALU AND	$A \leftarrow A \wedge B$	0110	ALU_ctrl=00, A_enable
ALU ADD	$A \leftarrow A + B$	0111	ALU_ctrl=10, A_enable
ALU OR	$A \leftarrow A \vee B$	1000	ALU_ctrl=01, A_enable
ALU SUB	$A \leftarrow A \bar{\wedge} B$	1001	ALU_ctrl=11, A_enable
B load from A	$B \leftarrow A$	1010	B_enable, B_MUX selects A
Memory write	$M \leftarrow B$	1011	MemWrite=1
A load from B	$A \leftarrow B$	1100	A_enable, A_MUX selects B
Booth multiply	MUL	1101	Booth_enable, A_enable
Branch	$PC \leftarrow B(AD)$	0100	PC_enable, MUX selects IR[10:0]
Halt	HLT	1111	Clock gated off via AND+NOT

6. ALU Design

The 22-bit ALU was designed in a previous lab assignment and reused here as a subcircuit. It supports four operations selected by a 2-bit control signal.

Table-6.1: ALU Control Signal

Control (2-bit)	Operation	F field that triggers it
00	AND $\bar{A} \wedge B$	0110
01	OR $\bar{A} \vee B$	1000
10	ADD $\bar{A} + B$	0111
11	SUB $\bar{A} \bar{\vee} B$	1001

The 2-bit ALU control is generated from the decoder outputs using two OR gates:

- Control bit 1 (MSB) = decoder_out7 (ADD) OR decoder_out9 (SUB)
- Control bit 0 (LSB) = decoder_out8 (OR) OR decoder_out9 (SUB)

These two bits are combined using a splitter and fed to the ALU control pin.

7. Conclusion

This project demonstrates the design of a minimal 22-bit computer system in Logisim, following the microprogrammed control architecture from Morris Mano's Computer System Architecture. The system implements a complete fetch-decode-execute cycle controlled entirely by a ROM-based microprogram.

The key design decisions were:

- A 22-bit word size applied consistently across all registers, buses, and memory
- A microprogrammed Control Unit using a 64-location control ROM (6-bit CAR)
- A 4-to-16 decoder for clean control signal generation from the F field
- The Booth's multiplication algorithm integrated as a subcircuit for the MUL instruction
- A hierarchical subcircuit structure (main $\rightarrow$ CPU $\rightarrow$ ControlUnit) keeping the top-level view clean

The system successfully executes memory-reference instructions (LDA, STO, ADD, SUB, AND, OR, BUN, MOV) and supports complex multiplication via Booth's algorithm. Programs are loaded directly into RAM and executed automatically when the clock runs, with no changes required to the ROM microprogram between different programs.